

Taming the Wildcards: Combining Definition- and Use-Site Variance

Shan Shan Huang^{1,2} and Yannis Smaragdakis²

¹ Georgia Institute of Technology
`ssh@cc.gatech.edu`

² University of Massachusetts, Amherst
`yannis@cs.umass.edu`

Abstract. Variance allows the safe integration of parametric and subtype polymorphism. Two flavors of variance, definition-site versus use-site variance, have been studied and have had their merits hotly debated. Definition-site variance (as in Scala and C#) offers simple type-instantiation rules, but causes fractured definitions of naturally invariant classes; Use-site variance (as in Java) offers simplicity in class definitions, yet complex type-instantiation rules that elude most programmers. We present a technique that combines the best of both worlds. Our technique *modularly infers* the definition-site variance of type parameters, while allowing use-site variance annotations on any type-instantiation. Applying our technique to six Java generic libraries (including the Java core library) showed that 46% of generic definitions have single-variance that can be inferred; 41% of method argument types can be relaxed through this inference, and 52% of existing wildcard annotations are unnecessary and can be elided. Our technique is implemented as a front-end extension of the Java type-checker, and is completely backward compatible.

1 Introduction

Variance mechanisms are the keystone of safe genericity in modern programming languages. Variance in an object-oriented type system attempts to reconcile the very different mechanisms of parametric and subtype polymorphism. Concretely, variance mechanisms aim to answer the question “under what conditions for type expressions *Exp1* and *Exp2* is $C\langle Exp1 \rangle$ a subtype of $C\langle Exp2 \rangle$?”

The conventional answer to this question has been *definition-site variance*: the definition of generic class $C\langle X \rangle$ determines its variance [1, 9, 13]. Depending on how the type parameter X is used, C can have one of four flavors of variance: it can be *covariant*, meaning that $C\langle S \rangle$ is a subtype of $C\langle T \rangle$ if S is a subtype of T ; it can be *contravariant*, meaning that $C\langle S \rangle$ is a subtype of $C\langle T \rangle$ if T is a subtype of S ; it can be *bivariant*, meaning that $C\langle S \rangle$ is always a subtype of $C\langle T \rangle$; or it can be *invariant*, meaning that $C\langle S \rangle$ and $C\langle T \rangle$ are never subtype-related for different types T and S .

An alternative approach has been introduced in the past decade: *Use-site variance* [17] interprets the definition of a generic class $C\langle X \rangle$ as the introduction

of 4 distinct types in the type system: the covariant, contravariant, bivariant, and invariant part of $C\langle X \rangle$. Each of these types only supports the methods compatible with each variance designation. For instance, the covariant part of $C\langle T \rangle$ would only support methods that would be safe to call on an object of type $C\langle S \rangle$ when S is a subtype of T . In this way, the use-site of a generic class declares what it can accept, with a vocabulary such as “C-of-any-subtype-of- T ”, and the type system checks that the use is indeed valid.

Use-site variance is a truly elegant idea. Producing automatically all different variance flavors from a single class definition is an approach of hard-to-dispute flexibility. The idea was quickly integrated in Java in the form of *wildcards* and it is widely used in standard Java libraries. Despite the conceptual elegance, however, the practical deployment of wildcards has been an unmitigated disaster. “Wildcards” has become a virtual synonym for a language design mess. (E.g., Josh Bloch’s presentation at Javapolis 2008 emphasized “We simply cannot afford another wildcards” [3].) The reason is that use-site variance results in conceptual complexity, requires anticipation of generality at all usage points, and postpones the detection of overly restrictive type signatures until their use.

For these pragmatic reasons, newer languages, such as Scala [20] and C# [15], have returned to and refined the tried-and-true approach of definition-site variance. Definition-site variance is hardly free of usability problems, however. For a class that is not purely covariant or contravariant, the only way to achieve full genericity is by introducing specialized interfaces that correspond to the class’s co-, contra-, and bivariant parts. Consequently, users have to remember the names of these interfaces, library designers must anticipate genericity, and a combinatorial explosion in the number of introduced interfaces is possible. (E.g., for a type $\text{Triple}\langle X, Y, Z \rangle$, we may need an interface for each of the 27 possible access combinations, such as “covariant with respect to X , contravariant with respect to Y and Z ”.)

In this paper, we present a language design that mitigates the tension between use-site and definition-site variance. Our language allows all existing Java use-site variance annotations (wildcards), but *infers* variance automatically for classes that are purely covariant or purely contravariant with respect to a type parameter—*without the need for its variance to be declared*. The inferred variance is then used to relax the use-sites of the class. This means that our solution is fully compatible with existing Java syntax and only changes the type system to make it more permissive in cases of purely variant types, so that exhaustive use-site annotation is unnecessary. We conducted a study of several large Java libraries and show that our technique has a large impact. More specifically, the contributions of our work are:

- We offer an inference technique for computing the definition-site variance of classes and interfaces. Unlike past work, our inference technique is *modular*: the variance of a generic class is purely determined by its own interface definition, *not* the code that uses the class.
- Although the inference of definition-site variance is well-understood for simple types, our inference algorithm is the first to work with type recursion and

wildcards. For instance, we know of no technique that would infer anything other than *invariance* for classes `C` and `D`, in the following example:

```
class C<X> {
  X    foo (C<? super X>    csx);
  void bar (D<? extends X> dsx) { ... }
}
class D<Y> {
  void baz (C<Y> cx) { ... }
}
```

Our inference algorithm correctly infers the most general variance: class `C` is *covariant* with respect to `X`, and class `D` is *contravariant* with respect to `Y`. The interaction of wildcards and type recursion is highly non-trivial. For instance, removing the “`? super`” from the type of argument `csx` would make both `C` and `D` be invariant.

- Our approach is backward compatible with Java, requiring changes only to the type system. Under our updated type system, many uses of variance annotations become redundant. All previously legal Java programs remain legal, although some newly type-correct programs might have been previously rejected.
- We conduct a large study to show how use-site and definition-site variance coexist fruitfully, even in code that was written with only use-site variance in mind! Past literature [12, 17] has largely assumed that single-variant type parameters are seldom used, thus promoting use-site variance for flexibility. We show that this is not the case—we find that 46% of classes and interfaces analyzed have single-variance type parameters. This is strong evidence for the need to combine definition- and use-site variance, since they both occur commonly. Furthermore, our inference algorithm obviates the need for many uses of wildcards: 52% of existing wildcard uses are rendered unnecessary.

In the rest of the paper, we discuss the background of variance mechanisms in Section 2, present the inference logic informally in Section 3, and give our formal algorithm in Section 4. Section 5 analyzes our experimental results, before discussing related work (Section 6) and concluding (Section 7).

2 Type-Checking Variance

The study of variance has a long history [1, 5–7, 9, 13, 17, 23]. It aims to provide safe conditions for subtyping between different instantiations of the same generic class or interface. Consider the following example:

```
class List<X> {
  void set(int i, X x) { ... }
  X    get(int i)      { ... }
}
```

If we naively assume that `List<S> <: List<T>` for any `S <: T` (where `<:` is the subtyping relation), the following ill-typed code could pass compile-time type-checking, and result in runtime errors:

```
List<Integer> intList = new List<Integer>();
List<Number> numList = intList;
numList.set(0, new Float(0.0f)); // Writing Float into Integer!
```

The key to safe parametric subtyping lies with the *variance* of type parameter X in the definition of $C<X>$. If X only appears *covariantly*, then it is always safe to assume that $C<S> <: C<T>$ if $S <: T$. If X only appears *contravariantly*, it is always safe to assume that $C<S> <: C<T>$ if $T <: S$. If X appears both co- and contravariantly, C is invariant: $C<S> <: C<T>$ only if $S = T$. This notion of variance is called *definition-site variance*.

Definition-site Variance. Languages supporting definition-site variance [15, 19, 20] require each type parameter to be declared with a variance annotation. For instance, Scala [20] requires $+$ for covariant type parameters, $-$ for contravariant type parameters, and invariance is the default. A well-established set of rules can then be used to verify that the type parameter only occurs with the declared variance within the definition of the generic.³ We provide an overview of these rules here, as they form the basis of both use-site variance and our inference.

Each typing position in a class or interface’s signature has an associated variance. For instance, method return and exception types, supertypes, and upper bounds of type parameters are covariant positions; method argument types and type parameter lower bounds are contravariant positions; field types are both co- and contravariant positions. Covariant positions *preserve* the variance of types that appear in them, whereas contravariant positions *reverse* the variance of the types that appear in them. The “reverse” of covariance is contravariance, and vice versa. The “reverse” of invariance is itself.

Thus, we can consider the occurrence of a type parameter to be initially covariant. This variance is then transformed by the position it appears in. For instance, consider the following interfaces:

```
interface ROList<+X> { X get(int i);      }
interface WOList<-Y> { void set(int i, Y y); }
```

In `ROList`, X appears as the return type of a method, which preserves the initial covariance of X . Thus, `ROList` is covariant in X ,⁴ as declared. Intuitively, `ROList` is a read-only list: it only supports retrieving objects. Retrieving objects of type T can be safely thought of as retrieving objects of any supertype of T . Thus, a read-only list of T s (`ROList<T>`) can always be safely thought of as a read-only list of some supertype of T s (`ROList<S>`, where $T <: S$). This is the exact definition of covariant subtyping.

Similarly, `WOList` is a write-only list, and is intuitively contravariant. Its definition supports this intuition: Y appears in the contravariant method argument position, which reverses its initial covariance, to contravariance. Thus, `WOList` is contravariant as declared.

³ We herein refer to generic classes and interfaces uniformly as “generics”.

⁴ For brevity, if it is clear from the context, we do not specify which type parameter a definition-site variance is with respect to.

When a type parameter is used to instantiate a generic, its variance is further transformed by the definition-site variance of that generic. For example:

```
interface SourceList<+Z> { void copyTo(WOList<Z> target); }
```

In `WOList<Z>`, the initial covariance of `Z` is transformed by the definition-site variance of `WOList` (contravariance). It is then transformed again by the contravariant method argument position. As a result, `Z` appears covariantly in this context, and `SourceList` is covariant in `Z`, as declared. Any variance transformed by invariance becomes invariance. Thus, if `Z` was used to parameterized an invariant generic, its appearance would have been invariant.

We have so far neglected to discuss *bivariance*: `C<X>` is bivariant implies that `C<S> <: C<T>` for any types `S` and `T`. Declaring a bivariant type parameter is not supported by any definition-site variant languages. This is not entirely surprising. For a type parameter to be bivariant, it must only appear bivariantly in a generic. This means either it does not appear at all, or it appears only as the type argument to instantiate other “bivariant” generics. If a type parameter does not appear in a generic’s signature at all, then it is useless to parameterize over it; if it is only used to instantiate other bivariant generics, it could just as well be replaced by any arbitrary type, since by definition, a bivariant generic does not care what type it is instantiated with. (For conceptual completeness, any variance transformed by bivariance—including invariance—becomes bivariance.)

Use-site Variance. An alternative approach to variance is *use-site variance* [6,17,23]. Instead of declaring the variance of `X` at its definition site, generics are assumed to be *invariant* in their type parameters. However, a type-instantiation of `C<X>` can be made co-, contra-, or bivariant using variance annotations.

For instance, using the Java wildcard syntax, `C<? extends T>` is a *covariant* instantiation of `C`, representing a type “C-of-some-subtype-of-T”. `C<? extends T>` is a supertype of all type-instantiations `C<S>`, or `C<? extends S>`, where `S <: T`. In exchange for such liberal subtyping rules, type `C<? extends T>` can only access those methods and fields of `C` in which `X` appears covariantly. In determining this, use-site variance applies the same set of rules used in definition-site variance, with the additional condition that the upper bound of a wildcard is considered a covariant position, and the lower bound of a wildcard a contravariant position.

For example, `List<? extends T>`, only has access to method “`X get(int i)`”, but not method “`void set(int i, X x)`”.

Similarly, `C<? super T>` is the contravariant version of `C`, and is a supertype of any `C<S>` and `C<? super S>`, where `T <: S`. Of course, `C<? super T>` has access only to methods and fields in which `X` appears contravariantly.

Use-site variance also allows the representation of the *bivariant* version of a generic. In Java, this is accomplished through the unbounded wildcard: `C<?>`. Using this notation, `C<S> <: C<?>`, for any `S`. The bivariant type, however, only has access to methods and fields in which the type parameter does not appear at all. In definition-site variance, these methods and fields would have to be factored out into a non-generic class.

A Comparison. Both approaches to variance have their merits and shortcomings. Definition-site variance enjoys a certain degree of conceptual simplicity: the generic type instantiation rules and subtyping relationships are clear. However, the class or interface designer must pay for such simplicity by splitting the definitions of data types into co-, contra-, and bivariant versions. This can be an unnatural exercise. For example, the data structures library for Scala contains immutable (covariant) and mutable (invariant) versions of almost every data type—and this isn’t even a complete factoring of the variants, since it does not include contravariant (write-only) versions of the data types.

The situation gets even more complex when a generic has more than one type parameter. In general, a generic with n type parameters needs 3^n interfaces to represent a complete variant factoring of its methods. Arguably, in practice, this is often not necessary.

Use-site variance, on the other hand, allows users of a generic to create co-, contra-, and bivariant versions of the generic on the fly. This flexibility allows class or interface designers to implement their data types in whatever way is natural. However, the users of these generics must pay the price, by carefully considering the correct use-site variance annotations, so that the type can be as general as possible. This might not seem very difficult for a simple instantiation such as `List<? extends Number>`. However, type signatures can very quickly become complicated. For instance, the following method signature is part of the Apache Commons-Collections Library:

```
Iterator<? extends Map.Entry<? extends K,V>>
    createEntrySetIterator(Iterator<? extends Map.Entry<? extends K,V>>)
```

Our approach can infer that `Iterator` is covariant, and `Map.Entry` is covariant in its first type parameter—without having to change the definition of either generics. Thus, the following signature in our system has exactly the same generality without any wildcards:

```
Iterator<Map.Entry<K,V>> createEntrySetIterator(Iterator<Map.Entry<K,V>>)
```

Furthermore, specifying the most general types proves to be challenging for even the most seasoned Java programmers: 61% of the types in method signatures of the Java core library (`java.*`) is overly specific. We will discuss the details of our findings in Section 5.

3 Inferring Variance

Our goal is to combine the positive aspects of use-site and definition-site variance, while mitigating their shortcomings. Class and interface designers should implement their data types in a way that is natural: e.g., if a `List` is supposed to support both reading and writing of data, then its implementation should include both kinds of methods, without having to be split into two variants. At the same time, programmers should enjoy simple type-instantiation rules *when possible*—definition-site invariance should not be the default. Furthermore, these

features should be offered without any programmer—the class designer or the class user—having to annotate his programs more than he already does.

Our solution is to *infer* the most general variance for each generic at its definition-site; at the same time, we allow use-site annotations on all type-instantiations.

As we discussed in the previous section, past literature has identified covariant and contravariant positions in a generic’s definition. Method returns and exception types, supertypes, upper bounds of type parameters, and the upper bound of a wildcard are covariant positions. Method argument types, lower bounds of wildcards, and type parameter lower bounds are contravariant positions. Field types are both co- and contravariant positions. (The classification has not been so explicit, but it is more-or-less understood by experts.) Thus, a straightforward inference algorithm would be: (a) examine what positions a type parameter appears in; (b) if all positions are covariant, designate the generic to be covariant with respect to the type parameter; (c) if all positions are contravariant, designate the generic to be contravariant with respect to the type parameter; (d) if positions are of both kinds, designate the generic to be invariant; (e) if the type parameter appears in no covariant or contravariant position, designate the generic to be bivariant with respect to the type parameter.

This raises the natural question: *How is our inference approach more advanced than the straightforward algorithm?*

The difference, and the technical contribution of this paper, lies in 1) the handling of recursive types, and 2) the integration of use-site variance annotations (wildcards) with definition-site variance to infer the most general variance.

The example from Section 1 nicely highlights the deficiencies in existing definition-site and use-site variance techniques, and showcases the interesting technical issues in our endeavor:

```
class C<X> {
  X    foo (C<? super X> csx);
  void bar (D<? extends X> dsx) { ... }
}
class D<Y> {
  void baz (C<Y> cx) { ... }
}
```

To infer the definition-site variance of `C`, we need to check the variance of `X`’s uses in `C`’s signature. `X` clearly appears covariantly as the return type of method `foo`. However, it also appears as argument types `C<? super X>` and `D<? extends X>`. Determining the variance of `X` in these types requires two types of information.

Recursive Variances. The variance of `X` is dependent upon the variances of `C<X>` and `D<Y>`. However, the variance of `C<X>` is precisely what we are trying to infer! Furthermore, the variance of `D<Y>` is dependent upon the variance `C<X>`, as well. Thus, we have two recursive variance dependencies.

Note that this issue is unique to the inference problem. *Traditional definition-site variance does not have to handle type-recursions: the variance of every*

generic is declared. We do not have to *infer* what the variance of `C<X>` is, only to *type check* that its signature is consistent with its declaration.

Of course, since the number of possible variances is finite, we could just turn the “inference” into an exhaustive search for a set of consistent variance annotations. This brute force method, however, has an exponential complexity with respect to the number of generics in the program, which is intractable even for trivial amounts of code. As we will show, our technique truly *infers* the variance annotations, in *polynomial time*.

Integrating Wildcards. How do the wildcard annotations affect the variance of `X`? Traditional use-site variance techniques would classify `X` to be contravariant in `C<? super X>`, and covariant in `D<? extends X>`. This means `X` occurs both co- and contravariantly in `C`. Thus, the inferred variance of `C` would be invariant. However, the most general variance for `C` is actually covariant.

The issue is that *traditional use-site variance rules assume invariance of both `C` and `D`, with respect to their type parameters*. Therefore, they conservatively assign co- and contravariance to `X` in `C<? super X>` and `D<? extends X>`. As we will show in Section 3.2, integrating the true definition-site variance of both `C` and `D` allows us to assign much more liberal variances to both of these uses of `X`.

Next we dive into the details of recursive variances and the integration of use-site and definition-site variance. We will conclude this section by applying our technique to infer the variance of `C` and `D` in this same example.

3.1 Recursive Variances

Recursive Variance Type 1: The following interface demonstrates a most simple form of recursive variance:

```
interface C1<X> { C1<X> foo1 (); }
```

The variance of `C1` depends on how `X` appears in its signature. The only appearance of `X` is in the return type of `foo1`, a covariant position, as the argument to `C1`. Thus, the variance of `X` in this appearance is its initial covariance, transformed by the variance of `C1`—the very variance we are trying to infer! This type of recursive variance essentially says that the variance of `C1` is the variance of `C1`, and thus can be satisfied by any of the four variances: co-, contra-, in-, or bi-variance. Without any more appearances of `X`, the most liberal form of variance for `C1` is bivarience.

If `X` does appear in other typing positions, however, the variance of its declaring generic is completely determined by the variance of these appearances:

```
interface C2<X> extends C1<X> { void bar2 (X x); }
interface C3<X> extends C1<X> { X bar3 (); }
```

The definition-site variance of `C2` is constrained by the variance of `C1`, as well as `X`’s appearance as a method argument type—a contravariant appearance. Since `C1`’s variance is completely unconstrained, `C2` is simply contravariant. Similarly,

`C3` is only constrained by `X`'s appearance as a method return type—a covariant appearance—and is thus covariant, as well.

The above pattern will be common in all our recursive variances. Without any constraints other than the recursive one, a generic is most generally bivariant. When other constraints are factored in, however, the real variance of `C1` can be understood informally as “can be either co- or contravariant”.

Recursive Variance Type 2: The next example shows a similar, but much more restrictive form of recursive variance:

```
interface D1<X> { void foo1 (D1<X> dx); }
```

The variance of `D1` is again recursively dependent on itself, only this time `X` appears in `D1<X>` which is a method argument. If a recursive variance did not impose any restrictions in a covariant position, why would it be any different in a contravariant position? Interestingly, the contravariance means that the variance of `D1` is the variance of `D1` transformed by the contravariance. This means the variance of `D1` must be the *reverse* of itself!

The only two variances that can satisfy such a condition are bi- and invariance. Again, without any other uses of `X`, `D1<X>` is most generally bivariant.

However, if `X` does appear either co- or contravariantly in combination with this type of recursive variance, the resulting variance can only be *invariance*:

```
interface D2<X> extends D1<X> { void bar2 (X x); }
interface D3<X> extends D1<X> { X bar3 (); }
```

In the above example, `X` appears contravariantly in `D2`, as the argument of `bar2`. At the same time, the variance of `X` must be the opposite of itself, as constrained by the recursive variance in supertype `D1`. This is equivalent to `X` appearing covariantly, as well. Thus, the only reasonable variance for `D2` is invariance. A similar reasoning results in the invariance of `D3`.

Thus, recursive variance type 2 can be understood informally as “cannot be either co- or contravariant” when other constraints are taken into account.

Recursive Variance Type 3: The following example shows yet a third kind of recursive variance:

```
interface E1<X> { E1<E1<X>> foo1 (); }
```

The variance of `E1` is the same as `X`'s variance in `E1<E1<X>>`. That is, the initial covariance of `X`, transformed by the variance of `E1`—twice. This type of recursive variance can, again, like the previous two, be satisfied by either in- or bivariance. However, the key insight is that, no matter whether `E1` is contra- or covariant, any variance transformed by `E1` twice (or any even number of times, for that matter) is always preserved. This is obvious if `E1` is covariant. If `E1` is contravariant, being transformed by `E1` twice means a variance is reversed, and then reversed again, which still yields a preserved variance. Thus, unless `E1` is bi- or invariant, `X` in `E1<E1<X>>` is always a covariant appearance.

Thus, when other appearances of X interact with this form of recursive variance, its informal meaning becomes “cannot be contravariant”. In other words, when this recursive variance is part of the constraints of a type, the type can be bivariant, covariant, or invariant. The following examples demonstrate this:

```
interface E2<X> extends E1<X> { void bar2 (X x); }
interface E3<X> extends E1<X> { X    bar3 ();    }
```

X appears contravariantly in $E2$, eliminating bivarience and covariance as an option for $E2$. However, X also appears in $E1<E1<X>>$ through subtyping, which means it cannot be contravariant. Thus, $E2$ is invariant.

In $E3$, X appears covariantly, and X in $E1<E1<X>>$ can still be covariant. Thus, $E3$ can safely be covariant.

Recursive Variance Type 4: The last kind of recursive variance is also twice constrained by itself. But this time, it is further transformed by a contravariance:

```
interface F1<X> { int foo1(F1<F1<X>> x); }
```

The variance of $F1$ is the same as X ’s variance in $F1<F1<X>>$, then transformed by the contravariant position of the method argument type. That is, X ’s initial covariance, transformed twice by the variance of $F1$, then reversed. Like all the other recursive variances, bi- and invariance are options. However, since the twice-transformation by any variance preserves the initial covariance of X in $F1<F1<X>>$, the transformation by the contravariance produces a contravariance. Thus, if $F1$ cannot be bivariant, it must be contravariant (or invariant).

In other words, along with other constraints, $F1$ has the informal meaning: “cannot be covariant”. For instance:

```
interface F2<X> extends F1<X> { void bar2 (X x); }
interface F3<X> extends F1<X> { X    bar3 ();    }
```

In $F2$, X appears contravariantly as a method argument. Combined with the recursive variance through subtyping $F1<X>$, $F2$ can be contravariant. In $F3$, however, X appears covariantly. With bivarience and contravariance no longer an option, the only variance satisfying both this covariant appearance and the recursive variance of $F1<F1<X>>$ is invariance. Thus, $F3$ is invariant in X .

General Case. The four types of recursive variances we introduced cover all cases of possible recursions. As we explained, being transformed by any variance an even number of times preserves the variance being transformed. Thus, the four types of recursive variance can be classified along two dimensions: the number of times the variance depends on its own transformation, and the number of times it is transformed by contravariance:

	odd # contravariances	even # contravariances
odd # self transformations	type 2	type 1
even # self transformations	type 4	type 3

The challenge in our inference is to expose these recursions. The recursions that appear in real programs are generally not direct self-references: They can be complex cycles over multiple class definitions. We use a simple “solving” approach to reduce them to the question of a direct self-reference. For instance, in the following interfaces, finding this recursive constraint is not at all obvious:

```
interface C<X> { D<X> foo() ; }
interface D<X> { void bar(C<E<X>> cfx); }
interface E<X> { C<X> baz(); }
```

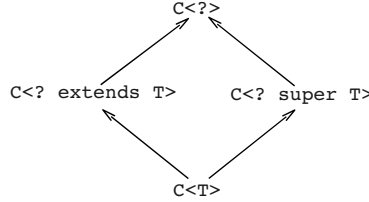
Following the chain, however, reveals that the variance of `C` is recursively dependent on itself twice, and transformed once by a contravariance—recursive variance type 4. (Since the example includes no constraints other than the recursive one, the result is still that all generics are bivariant.)

We show the precise algorithm to uncover these recursions in Section 4.

3.2 Incorporating Use-site Variance Annotations

Another novel aspect of our work is the interaction between definition-site variance inference and explicit use-site variance annotations. Combining information about definition-site variance with use-site annotations can lead to more general types, which, in turn, lead to more general inferred definition-site variance.

The familiar type hierarchy created by annotating an *invariant* generic `C<X>` with use-site variance annotations is as follows:



Traditionally, use-site variance techniques classify the use of `T` in `C<? extends T>` as covariant, and the use of `T` in `C<? super T>` as contravariant. However, these classifications are made under the assumption that `C` is *invariant* in its type parameter.

Incorporating knowledge about the definition-site variance, however, we can arrive at a more general classification. `T` is *at least* covariant in `C<? extends T>`, and *at least* contravariant in `C<? super T>`. Depending on the definition-site variance of `C`, `T` could even be bivariant.

Consider a `C<X>` that is definition-site covariant in `X`. This means that `X` only appears covariantly in `C`. Since the covariant wildcard type `C<? extends T>` has access to all members of `C` in which its type parameter appears covariantly, `C<? extends T>` has access to *all* members of `C`. Thus, for a covariant `C`, the types `C<T>` and `C<? extends T>` collapse into equivalent types. Type `C<? super T>`, however, only has access to members in which the type parameter appears contravariantly. By definition, there are no such members in `C`. Thus, like `C<?>`, `C<? super T>` can

only access members of C in which the type parameter does not appear at all. In this sense, $C<?>$ and $C<? \text{ super } T>$ collapse into equivalent types. Thus, T in $C<? \text{ super } T>$ can be considered a *bivariant* position—regardless of what T is, $C<? \text{ super } T>$ has access to the exact same methods.

Similarly, if C is contravariant, $C<T>$ and $C<? \text{ super } T>$ are equivalent types, and $C<? \text{ extends } T>$ and $C<?>$ are equivalent types. T in $C<? \text{ extends } T>$ can be considered a bivariant position.

Thus, we arrive at the following collapsed type hierarchies when wildcards are used together with generics with single definition-site variance. ($C<+T>$ denotes the instantiation of $C<+X>$ with T , and $C<-T>$ the type-instantiation of $C<-X>$):

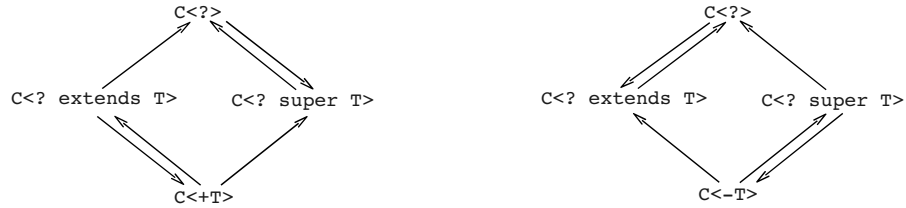


Fig. 1. Integration of use-site and definition-site variance.

3.3 Inferring the Example

With our full arsenal of techniques discussed, we can now go back to our example and see how we can infer covariance for C , and contravariance for D .

```
class C<X> {
  X    foo (C<? super X> csx);
  void bar (D<? extends X> dsx) { ... }
}
class D<Y> {
  void baz (C<Y> cx) { ... }
}
```

The variance of $C<X>$ with respect to X is a combination of (at least as restrictive as either) the variance constraints from methods `foo` and `bar`. In `foo`, X appears in $C<? \text{ super } X>$, which means it has at least a contravariance, but possibly a more general variance (depending on the variance of $C<X>$, recursively), and then transformed once by the contravariance of the position (argument type). The recursive variance constraint is of type 2, so the only variance that can satisfy it is either bivariance or invariance. Bivariance is impossible, however: the X in the return type already constrained the variance of C to at most covariant. Therefore, the total variance of $C<? \text{ super } X>$ is just the contravariance of the wildcard. Transformed by the contravariant position of method argument type,

we get a covariant appearance of X . So far, we have X appearing covariantly in both the return and argument types of method `foo`.

Now let us take a look at method `bar`. Again, the variance of X transformed by $D<? \text{ extends } X>$ type is at least covariant, depending on what the variance of $D<Y>$ is. The variance of $D<Y>$ is the opposite of the variance of $C<X>$. However, this variance must be transformed another time by the contravariance position of its appearance as `bar`'s argument. Thus, we have recursive variance type 1: single recursion, even number of contravariant transformations. This variance is fully general and not constrained at all.

Thus, combining the constraints from methods `foo` and `bar`, $C<X>$ is covariant in X . Knowing this, it is easy to determine that $D<Y>$ is contravariant in Y .

A subtlety that we have ignored in this section, for simplicity, is that variance depends on method visibility and on the “observer” type. For instance, if `foo` and `bar` were `private` methods, `C` would be invariant to the observer `D`, as `D` cannot access any method of `C`. Our full algorithm, presented next, accounts for different observers.

4 Inference Algorithm

In this section, we describe the formal framework underlying our inference. The goal of our algorithm is to infer the most general variance annotation possible for a type parameter, at the definition site of that parameter.

4.1 The Basics: Variance Lattice and Transformation

We present the “generality” of variance annotations as a partial order depicted by the lattice in Figure 2. The dotted arrows indicate the directions for the familiar $\sqcup$ and $\sqcap$ lattice operations. The join ($\sqcup$) operation returns the least upper bound of its inputs: a variance that is *more* general than or equal to the two input variances. The meet ($\sqcap$) operation returns the greatest lower bound of its inputs: a variance that is *less* general than or equal to the two input variances.

Along with the basic variance annotations for co-, contra-, in-, and bivarience (+, -, o, and * respectively), we introduce notations for the recursive variances described in the previous section. A recursive variance σ_f^r has r number of type recursions, and f number of contravariances, where r and f can be + or -: + means even, and - means odd. Thus, σ_+^- and σ_-^- are recursive variances produced by odd numbers of type recursions, and transformed by an even (odd, respectively) number of contravariances. They correspond to recursive variance types 1 and 2 described in Section 3, respectively. Similarly, σ_+^+ and σ_-^+ are both recursive variances produced by even numbers of recursions, and transformed by an even (odd, respectively) number of contravariances. They correspond to recursive variance types 3 and 4, respectively.

The positions of these recursive variances in the lattice reflect our discussion in Section 3: σ_+^- places no constraints on the variance of a type parameter, thus is more general than any variance; σ_-^- , on the other hand, is stricter than all

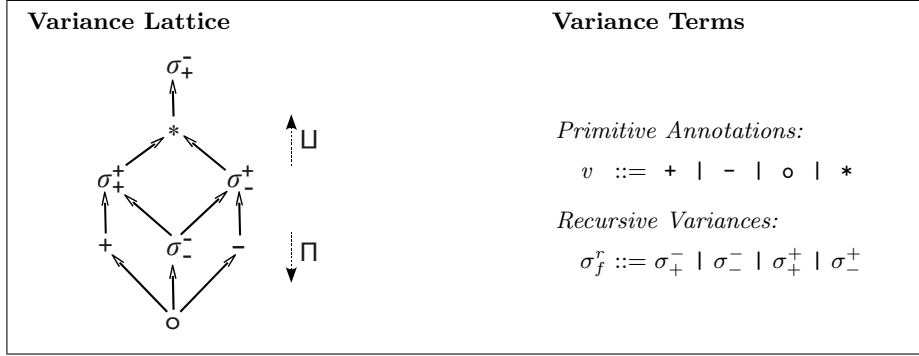


Fig. 2. Variance lattice.

recursive references, and has no relationship to $+$ or $-$; the σ_+^+ variance is more general than $+$, and the σ_-^+ variance is more general than $-$.

Inspecting the lattice, one might ask why σ_+^- is not the same as $*$. Certainly, the two variances are indistinguishable in the $\sqcap$ and $\sqcup$ operations on the lattice. They differ, however, in an operation ($\otimes$) external to the lattice, that we define next. For the same reason, σ_-^- is not the same as $\circ$.

Definition of variance transformation: $\otimes$			
$+$	$\otimes v = v$	$+$	$\otimes \sigma_f^r = \sigma_f^r$
$-$	$\otimes - = +$	$-$	$\otimes \sigma_{f_0}^r = \sigma_{f_1}^r$, where $f_1 = f_0 \otimes -$
$\circ$	$\otimes \circ = \circ$	$\circ$	$\otimes \sigma_f^r = \circ$
$*$	$\otimes v = *$	$*$	$\otimes \sigma_f^r = *$
		$\sigma_{f_0}^{r_0} \otimes \sigma_{f_1}^{r_1} = \sigma_{f_2}^{r_2}$, where $r_2 = r_0 \otimes r_1$, $f_2 = f_0 \otimes f_1$	

Fig. 3. Variance transformation.

We define an associative and commutative operator, $\otimes$, for the transformation of one variance by another (Figure 3). This operator is central in our inference. The total variance of a type parameter in a type expression is computed from individual variances, connected via $\otimes$. For instance, if type parameter X in class C appears in the type $D\langle E\langle X \rangle \rangle$, as a method argument, then the variance of X is computed by connecting through $\otimes$ the variance of the method argument position ($-$), with the variance of D , whatever this might be, and with the variance of E . (D or E may also refer to type C , introducing recursive variances.)

Transformations between the basic variance annotations are straightforward: $+$ preserves the variance transformed by it, whereas $-$ reverses it. Either $+$, $-$, or $\circ$, transformed by $\circ$, becomes $\circ$. Anything transformed by $*$ becomes $*$.

The transformations on the recursive variances require a bit more thought. When a recursive variance is transformed by $+$, neither its number of recursions

or contravariances change. Thus, $+ \otimes \sigma_f^r = \sigma_f^r$. The $-$ operator, however, does change the number of contravariances in $\sigma_{f_0}^r$: if f_0 was $+$ before (indicating an even number of contravariances), f_1 becomes $-$, indicating an odd number of contravariances; and vice versa. Similar reasoning applies to two recursive variances transforming each other: both the number of recursions and the number of contravariances may change. As with the basic variance annotations, a recursive variance transformed by $\circ$ (or $*$) is $\circ$ (or $*$, respectively).

Notational Conventions. In this section, we use the following notational conventions: C, D denote constant class names; X, Y, Z denote type variables; S, T, U, V denote types; We use the shorthand $\bar{T}$ for a sequence of types $T_0, T_1, \dots, T_n$. We use $\prod_{i=1}^n V_i$ to denote $V_1 \sqcap \dots \sqcap V_n$. Similarly, $\bigsqcup_{i=1}^n V_i$ denotes $V_1 \sqcup \dots \sqcup V_n$.

For expository simplicity, we assume there is only one type parameter per generic. However, our formalization can be easily generalized to include multiple type parameters. We also do not discuss type variables in the bounds of other type variables, or inner classes. It is simple to formalize the rules we laid out in the previous section regarding these elements in the framework that follows.

4.2 Variance Equation Construction

We use $dvar(X, C\langle X \rangle, T_o)$ to denote the definition site variance of the generic $C\langle X \rangle$, with respect to X , as observed by type T_o . T_o has at least **protected** access to members of $C\langle X \rangle$. If T_o is a type-instantiation of $C\langle X \rangle$, it also has **private** access, as well. We assume a standard syntax for classes, $C ::= \text{class } C\langle X \rangle \text{ extends } \bar{S} \{ \bar{F} \bar{M} \}$. As we described in Section 3, the variance of $C\langle X \rangle$ depends on X 's appearance in $C\langle X \rangle$'s fields (F_1 through F_n), methods (M_1 through M_m), and supertypes (S_1 through S_l). We formally define $dvar(X, C\langle X \rangle, T_o)$ in Figure 4.

Definition-site variance of $C\langle X \rangle$:

$$dvar(X, C\langle X \rangle, T_o) = \prod_{i=1}^n fvar(X, F_i, T_o) \sqcap \prod_{i=1}^m mvar(X, M_i, T_o) \sqcap \prod_{i=1}^l tvar(X, S_i, T_o)$$

Fig. 4. Constraint for definition-site variance.

$fvar(X, F_i, T_o)$, $mvar(X, M_i, T_o)$, and $tvar(X, S_i, T_o)$ represent the variance of X in a field, method, and supertype, respectively. They are defined in Figure 5. Thus, the equation in Figure 4 says that the definition-site variance of $C\langle X \rangle$ is the most specific of X 's variance in each of $C\langle X \rangle$'s members (methods and fields) and supertypes, where the “most specific” variance is produced by the $\sqcap$ operation.

Variance Produced by Field Declarations ($fvar$). If a field F is not visible from type T_o , it induces no variant constraint on X . Thus, the variance of X in such a field is $*$. If F is visible, however, the variance of X depends on how it

Variance of X in Field Declarations:

$$fvar(X, F_i, T_o) = \begin{cases} * & \text{if not } visible(F, T_o) \\ tvar(X, T, T_o) \sqcap (- \otimes tvar(X, T, T_o)) & \text{if } F = T \text{ f;} \end{cases}$$

Variance of X in Method Declarations:

$$mvar(X, M, T_o) = \begin{cases} * & \text{if not } visible(M, T_o) \\ tvar(X, R, T_o) \sqcap \prod_{j=1}^n (- \otimes tvar(X, A_j, C\langle X \rangle)) \sqcap \prod_{j=1}^m tvar(X, E_j, T_o) & \text{if, in class } C, M = R \text{ m } (\bar{A}) \text{ throws } \bar{E} \{ \dots \} \end{cases}$$

Variance of X in Type-instantiation S :

$$tvar(X, S, T_o) = \begin{cases} * & \text{if } X \text{ does not appear in } S \\ + & \text{if } S = X \\ dvar(Y, C\langle Y \rangle, T_o) \otimes tvar(X, U, T_o) & \text{if } S = C\langle U \rangle \\ + \sqcup (dvar(Y, C\langle Y \rangle, T_o) \otimes tvar(X, U, T_o)) & \text{if } S = C\langle ? \rangle \text{ extends } U \\ - \sqcup (dvar(Y, C\langle Y \rangle, T_o) \otimes tvar(X, U, T_o)) & \text{if } S = C\langle ? \rangle \text{ super } U \end{cases}$$

Note: for the last three rules, assume definition `class C<Y> { ... }`

Fig. 5. Variance constraint induced by field and method declarations.

is used in the type of F , T . We use $tvar(X, T, T_o)$ to denote the variance of X in the type-instantiation T , as observed from T_o . The definition of $tvar$ (Figure 5) will be explained in detail shortly. For the moment, the key thing to note is that the variance of X is the “meet” of both X ’s variance in T , *and* X ’s variance in T transformed by $-$. Recall that a field type position is both covariant and contravariant, Thus, the variance of X must be the “meet” of its variance in T *preserved*, and its variance in T *reversed*.

Variance Produced by Method Declarations ($mvar$). Similar to the variance induced by field declarations, if a method M is not visible to observer T_o , the variance of X in M is $*$. Otherwise, the variance of X by in M is the meet of X ’s variance in the return and exception types, *and* the reversed variance of X in the argument types. The key item of note here is that the observer type changes for the variance produced by the arguments. Instead of being observed from T_o , the variance of X in the method arguments is observed from the declaring type $C\langle X \rangle$ itself—the most permissive observer for this method. This ensures that when an argument type is an instantiation of $C\langle X \rangle$, its `private` members are visible and accounted for in the variance inference.

Variance in Type Instantiations ($tvar$). $tvar(X, S, T_o)$ denotes the variance of X in the type S , observed by T_o . The first two rules are straight-forward. The first rule says that if X does not appear in S at all, then its variance in type S , is $*$. For instance, the variance of X in either type `Object` or $C\langle ? \rangle$ is $*$. The second rule

says that the variance of X as itself is $+$ —this is consistent with our description in the last section, that we assume the initial variance of a type parameter to be covariant.

Variance in Parameterized Types. The third rule defines the variance of X when S is a parameterized type, $C\langle U \rangle$, in which X may be the type argument U itself, or it may be a type argument in U . The variance of X , then, is the variance of X in U , transformed ($\otimes$) by the *definition-site variance* of the generic X instantiates.

For example, `WOList` is contravariant. Thus, the variance of X in the type instantiation `WOList<X>` is the definition-site variance of `WOList`, $-$, transformed by X 's variance in X , $+$. Thus, $- \otimes + = -$. The variance of X in `WOList<X>` is $-$.

For a more involved example, what is the variance of X in `WOList<WOList<X>>`? It is the definition-site variance of `WOList`, $-$, transformed by the variance of X in `WOList<X>`: $- \otimes (- \otimes +) = +$. Thus, the variance of X in `WOList<WOList<X>>` is $+$.

Interaction with Wildcards. When X is part of a wildcard type-instantiation, the variance of X must be *at least* as general as the variance indicated by the wildcard. (Recall that wildcards limit the methods visible to the type, thus making it more general.) For instance, in `C<? extends U>`, the variance of X must be at least $+$. Conversely, in `C<? super U>`, the variance of X must be at least $-$. We then use the “join” operator to obtain the variance that is the upper bound of the wildcard variance, and the variance of X in the non-wildcard part, $C\langle U \rangle$.

For an example, the variance of X in `WOList<? super X>`, is the “join” of the wildcard variance, $-$, with X 's variance in `WOList<X>`: $-$. This results in $-$. Intuitively, if `WOList` is contravariant already, adding a contravariant wildcard does not provide any more relaxed subtyping rules for this type. Thus, the type `WOList<? super X>` collapses into the equivalent type of `WOList<X>`. However, the variance of X in type `WOList<? extends X>` is $+ \sqcup - = *$. This illustrates that for definition-site contravariant types, the *covariant* wildcard type collapses into the equivalent type of the unbounded wildcard type, `WOList<?>`.

4.3 Solving the Variance Equations

Figure 6 shows the syntax of the definition-site variance equation we construct using the rules from Section 4.2. If the terms in the $\otimes$ operations, P , involve only primitive variance annotations, then the equation can be easily solved. Even when P refers to the definition-site variance of a generic, the equation can still be easily solved by substituting the definition of that variance. The difficult part is when the referenced definition-site variance is the exact variance we are attempting to solve, or is dependent upon the variance we are trying to solve.

The challenge then, is first, to expose such recursive dependencies, and secondly, to introduce the right kind of recursive variances. We employ a set of normalization rules for these purposes.

Syntax of Equation:

$$\begin{array}{ll}
DV ::= V_d = T \sqcap \dots \sqcap T & T ::= P \otimes \dots \otimes P \\
V_d ::= dvar(\mathbf{X}, \mathbf{C}\langle\mathbf{X}\rangle, \mathbf{T}_o) & P ::= v \mid V_d \mid v \sqcup P
\end{array}$$

Fig. 6. Syntax of variance equation.

Figure 7 illustrates our normalization rules. The normalization operation $\llbracket \cdot \rrbracket_{\bar{V}_d}$ is parameterized by a set of definition-site variances, $\bar{V}_d$.⁵ We use $\bar{V}_d$ to keep track of the dependencies we have.

Normalization Rules:

$$\begin{array}{ll}
\frac{}{\llbracket T_0 \sqcap \dots \sqcap T_n \rrbracket_{\bar{V}_d} = \llbracket T_0 \rrbracket_{\bar{V}_d} \sqcap \dots \sqcap \llbracket T_n \rrbracket_{\bar{V}_d}} & (SPLIT-\sqcap) \\
\frac{}{\llbracket T_0 \sqcup \dots \sqcup T_n \rrbracket_{\bar{V}_d} = \llbracket T_0 \rrbracket_{\bar{V}_d} \sqcup \dots \sqcup \llbracket T_n \rrbracket_{\bar{V}_d}} & (SPLIT-\sqcup) \\
\frac{T = T_0 \otimes (T_1 \sqcap T_2)}{\llbracket T \rrbracket_{\bar{V}_d} = \llbracket T_0 \otimes T_1 \rrbracket_{\bar{V}_d} \sqcap \llbracket T_0 \otimes T_2 \rrbracket_{\bar{V}_d}} & (\otimes-\sqcap) \\
\frac{T = T_0 \otimes (T_1 \sqcup T_2)}{\llbracket T \rrbracket_{\bar{V}_d} = \llbracket T_0 \otimes T_1 \rrbracket_{\bar{V}_d} \sqcup \llbracket T_0 \otimes T_2 \rrbracket_{\bar{V}_d}} & (\otimes-\sqcup) \\
\frac{T = t_0 \otimes \dots \otimes t_n \quad \text{for all } 0 \leq i \leq n, \ t_i = v \text{ or } \sigma_f^r \text{ or } V_d}{\llbracket T \rrbracket_{\bar{V}_d} = \llbracket t_0 \rrbracket_{\bar{V}_d} \otimes \dots \otimes \llbracket t_n \rrbracket_{\bar{V}_d}} & (\otimes-BASE) \\
\frac{}{\llbracket v \rrbracket_{\bar{V}_d} = v} & (N-VAR) \\
\frac{}{\llbracket \sigma_f^r \rrbracket_{\bar{V}_d} = \sigma_f^r} & (N-CYCLE) \\
\frac{V_d \notin \bar{V}_d \quad V_d = T_0 \sqcap \dots \sqcap T_n}{\llbracket V_d \rrbracket_{\bar{V}_d} = \llbracket T_0 \sqcap \dots \sqcap T_n \rrbracket_{\bar{V}_d : V_d}} & (DV) \\
\frac{V_d \in \bar{V}_d}{\llbracket V_d \rrbracket_{\bar{V}_d} = \sigma_+^-} & (INTRO-\sigma)
\end{array}$$

Fig. 7. Syntax, normal form, and normalization rules.

Rules *SPLIT*- $\sqcap$ and *SPLIT*- $\sqcup$ deconstruct the $\sqcap$ and $\sqcup$ connectives into independently normalizable parts. Rules $\otimes$ - $\sqcap$ and $\otimes$ - $\sqcup$ distribute the $\otimes$ operator over $\sqcap$ and $\sqcup$, respectively. Rule $\otimes$ -*BASE* says that if each operand of $\otimes$ is either a primitive or recursive variance, or a reference to a definition-site variance, then each operand can be separately normalized. Rules *N-VAR* and *N-CYCLE* simply say that primitive and recursive variances normalize to themselves.

⁵ We overload the overline syntax to represent a set here.

The interesting rules are *DV* and *INTRO-σ*. Rule *DV* says that if a definition-site variance is to be normalized, *and* it does not recursively depend on itself, i.e., it is not a member of the dependent variance set $\bar{V}_d$, then its normalization is the right-hand-side of its definition constructed using the rules in Section 4.2. The important part of this rule is that V_d itself is added to $\bar{V}_d$, as part of the recursive dependencies of $T_0 \sqcap \dots \sqcap T_n$. This addition enables us to find the recursive dependencies in any of the $T_i (0 \leq i \leq n)$ or *their subterms*. We then use rule *INTRO-σ* to introduce recursive variances.

Rule *INTRO-σ* says that if V_d is a definition-site variance that recursively depends on itself, then σ_+^- is introduced to replace V_d . Other types of σ_f^r 's are only derived through evaluating σ_+^- transformed by other variances. This evaluation, however, does not take place until normalization is complete. The normalization, in particular rules $\otimes$ - $\sqcap$ and $\otimes$ - $\sqcup$, are crucial in exposing σ_+^- to all the transformations affecting it.

Note that, if $V_d = dvar(\mathbf{x}, \mathbf{C} \langle \mathbf{x} \rangle, \mathbf{T}_o)$, and $dvar(\mathbf{x}, \mathbf{C} \langle \mathbf{x} \rangle, \mathbf{T}'_0) \in \bar{V}_d$, $V_d \in \bar{V}_d$ only if $\mathbf{T}_o = \mathbf{T}'_0$. This means that if the observer type of a definition-site variance changes, then the whole definition gets substituted into the term to be normalized—this gives us a chance to re-evaluate variances of $\mathbf{x}$ in **private** members.

The Algorithm. Solving the definition-site variance is then a simple application of the normalization rules. The algorithm is presented in Figure 8.

We begin normalization with $\bar{V}_d$ being the empty set. After normalization, we apply (implemented as *evaluate*) $\otimes$ as defined in Figure 3 and the $\sqcup$ and $\sqcap$ as defined by the lattice in Figure 2, to arrive at a final variance.

```

DefVar( $\mathbf{x}, \mathbf{C} \langle \mathbf{x} \rangle, \mathbf{T}_o$ ) =
   $V := \text{normalize}(dvar(\mathbf{x}, \mathbf{C} \langle \mathbf{x} \rangle, \mathbf{T}_o), \emptyset)$ 
   $v := \text{evaluate}(V)$ 
  if ( $v == \sigma_f^r$ ) return  $*$ 
  else return  $v$ 

```

Fig. 8. Solving the definition-site variance.

Note that, if the final variance is a recursive variance, then we return bivariance instead. This means that the variance of $\mathbf{x}$ in $\mathbf{C} \langle \mathbf{x} \rangle$ is constrained by nothing but its own variance (possible reversed by contravariance). As we explained in Section 3, such recursive variances are always satisfied most generally by $*$.

Next, we apply this algorithm to our example from Section 1, and infer the definition site variance of $\mathbf{c}$. This mirrors formally the informal reasoning of Section 3.3 on the same example. Assuming the observer **Object**, we have the following two equations, according to the rules in Figure 4 and 5:

Figure 9, lists the step-by-step normalization and evaluation of operations $\sqcup$, $\sqcap$, and $\otimes$, which finally lead to the resulting definition-site variance of $+$ for $\mathbf{c}$. (We use $dv_{\mathbf{C} \langle \mathbf{x} \rangle}$ as a shorthand for $dvar(\mathbf{x}, \mathbf{C} \langle \mathbf{x} \rangle, \mathbf{Object})$, and $dv_{\mathbf{D} \langle \mathbf{y} \rangle}$ as a shorthand for $dvar(\mathbf{y}, \mathbf{D} \langle \mathbf{y} \rangle, \mathbf{Object})$.)

$$\begin{aligned}
dvar(X, C<X>, Object) &= (+ \sqcap (- \otimes (- \sqcup dvar(X, C<X>, Object)))) \sqcap && \text{(method foo)} \\
&(- \otimes (- \sqcup dvar(Y, D<Y>, Object))) && \text{(method bar)} \\
dvar(Y, D<Y>, Object) &= - \otimes dvar(X, C<X>, Object) && \text{(method baz)}
\end{aligned}$$

Normalizing $dv_{C<X>}$:	
	$\llbracket dv_{C<X>} \rrbracket_{\emptyset}$
By <i>DV</i> :	$\frac{\llbracket (+ \sqcap (- \otimes (- \sqcup dv_{C<X>}))) \sqcap (- \otimes (+ \sqcup dv_{D<Y>})) \rrbracket_{\{dv_{C<X>}\}}}{\quad}$
By $\otimes$ - $\sqcup$, $\otimes$ -BASE:	$\frac{+ \sqcap ((- \otimes -) \sqcup - \otimes \llbracket dv_{C<X>} \rrbracket_{\{dv_{C<X>}\}}) \sqcap ((- \otimes +) \sqcup (- \otimes \llbracket dv_{D<Y>} \rrbracket_{\{dv_{C<X>}\}}))}{\quad}$
By <i>INTRO-σ, DV</i> :	$\frac{+ \sqcap ((- \otimes -) \sqcup (- \otimes \sigma_+^-)) \sqcap ((- \otimes +) \sqcup (- \otimes \llbracket - \otimes dv_{C<X>} \rrbracket_{\{dv_{C<X>}, dv_{D<Y>}\}}))}{\quad}$
By <i>INTRO-σ</i> :	$+ \sqcap ((- \otimes -) \sqcup (- \otimes \sigma_+^-)) \sqcap ((- \otimes +) \sqcup (- \otimes (- \otimes \sigma_+^-)))$
Evaluating operations:	
By $\otimes$ definition:	$\frac{+ \sqcap (+ \sqcup \sigma_-^-) \sqcap (- \sqcup \sigma_+^-)}{\quad}$
By lattice:	$\frac{+ \sqcap \sigma_+^+ \sqcap \sigma_+^-}{\quad}$
By lattice:	$+$

Fig. 9. Inferring variance of X in $C<X>$, from Section 1

Termination. The normalization rules in Figure 7 are in syntax-directed form. Thus, we use $m = n - |\bar{V}_d|$ as a measuring function, where n is the number of generic classes and interfaces in the program, and $|\bar{V}_d|$ the size of $\bar{V}_d$. Essentially, m measures the maximum number of variance definitions left to be expanded. Since we only add to $\bar{V}_d$, this number is bound to decrease with the application of the rules, to a minimum of 0.

Soundness. We can formally prove the soundness of our inference algorithm by enhancing an existing type framework [6, 17] with our inference, and prove soundness from first principles. An alternative is to provide a translation from our input program to one with our inference results expressed as use-site variance, and prove that the translated program is always well-typed. For instance, if we infer that class $C<X>$ is covariant in X , we can replace all *uses* of type $C<T>$ with $C<? \text{ extends } T>$, and similarly for contravariance and bivarience. (Some syntactic simplification is necessary: $C<? \text{ extends } ? \text{ super } T>$ becomes just $C<?>$.) If we prove that the program resulting from this transformation is always well-typed (per the standard type rules for use-site variance), then our inference is sound.

A formal proof of soundness for our technique is future work. However, we empirically verified the correctness of our inference using the technique outlined above. That is, when we report statistics on the inferred definition-site variance of classes in large Java libraries, we have confirmed our inference by (automatically) rewriting the libraries to use the inferred variance in all class use sites, and the resulting programs pass the Java type checker. (There is a corner case, of variance

over captured wildcards, where the Java type checker is overly conservative—we had to verify such instances, just 4 in the entire Java libraries, by hand.)

5 Analysis of Impact

To measure the impact of our approach, we ran our inference algorithm over 6 Java libraries, the largest of which is the core Java library from Sun’s JDK 1.6, i.e., classes and interfaces in the packages of `java.*`. The remainder of the libraries are JScience [10], a Java library for scientific computing; Google Collections [4], GNU Trove [14], and Apache Commons-Collection [2], three Java data structures libraries; and JPaul [21], a library supporting program analysis. Together, these libraries define 4,241 classes and interfaces, out of which 572 are generics. These generics declare 753 type parameters—i.e., some of the generics declare more than one type parameter.

We ran our algorithm under two different conditions. First, we took the libraries as they are, assuming that the programmers had accurately specified their interfaces (i.e., if a method argument type is `List<Number>`, then we assume that the body of the method truly uses `List` both co- and contravariantly). The *un-highlighted rows* of Figure 10 show the results from our first analysis.

The “Classes” columns show the percentage of classes (or interfaces) that are inferred by our algorithm to be invariant versus variant. An invariant class is invariant in all of its type parameters, whereas a variant class is variant in at least one of its type parameters. The “Type Params.” columns show a finer breakdown of the variance of type parameters. These two categories of numbers do not differ dramatically. Roughly 27% of the type parameters are variant, whereas 32% of classes or interfaces are variant in at least one type parameter. This means that about 1/3 of the generics defined should be allowed to enjoy general variant subtyping without users having to annotate them with wildcards.

The last two columns illustrate the burden of default invariant subtyping in Java, and the benefits of our approach. “Unnecessary Wildcards” shows the percentage of wildcards in method signatures that are unnecessary in *our system*, based on the *inferred definition-site variance* their generics. For instance, given that our technique infers interface `java.util.Iterator<E>` to be covariant, all instantiations of `Iterator<? extends T>`, for any `T`, are unnecessary. This number shows that, using our technique, 37% of the current wildcard annotations can be eliminated without sacrificing either type safety or the generality of types!

The “Over-specified Method” column lists the percentage of method arguments that are overly specific *in the Java type system*, based on the inferred definition-site variance of their generics. For instance, given that the inferred definition-site variance of `Iterator<E>` is covariant, specifying a method argument with type `Iterator<T>`, instead of `Iterator<? extends T>`, is overly specific, since the Java type system would preclude safe invocations of this method with arguments of type `Iterator`-of-some-subtype-of-`T`. Note that this percentage is *purely based on the inferred definition-site variance of the arguments’ types, not on analysis of the arguments’ uses in the bodies of methods*. This modular anal-

Library	# classes	# generics	Classes		Type Params.		Unnecessary Wildcards	Over-specified Methods
			inv.	variant	inv.	variant		
java.*	3255	133	62%	38%	66%	34%	22%	18%
			56%	44%	60%	40%	22%	61%
JScience	136	44	61%	39%	64%	36%	89%	7%
			57%	43%	58%	42%	89%	7%
Apache Coll.	426	209	70%	30%	75%	25%	51%	42%
			56%	44%	64%	36%	84%	58%
GNU Trove	246	48	63%	37%	63%	37%	0%	53%
			25%	75%	25%	75%	100%	53%
Google Coll.	92	65	75%	25%	81%	19%	55%	15%
			54%	46%	60%	40%	65%	50%
JPaul	86	73	77%	23%	78%	22%	0%	31%
			59%	41%	62%	38%	77%	69%
Total	4241	572	68%	32%	73%	27%	37%	19%
			54%	46%	59%	41%	52%	41%

■ Analysis with modified method signatures

Fig. 10. Inference results

ysis makes the results even more surprising: 19% of methods are over-specified. This means that *in our system*, 19% of the methods could be used in a much more liberal, yet still type-safe fashion, without library designers having to annotate the argument types carefully with wildcards.

Of course, the over-specified methods in Figure 10 are not *all* the over-specified methods in these libraries. Argument types of truly *invariant* generics may actually be used in a purely co- or contravariant manner in method bodies—e.g., just because a method is accepting a list, does not mean that it both reads and writes to it. Our first analysis conservatively looked at method signatures only. However, the results of our conservative analysis inspired us to conduct a second analysis, where we searched for over-specified method arguments by analyzing the bodies of the methods. If a method body only uses an argument in a co- or contravariant manner, we modify the method signature to reflect this variance. (This analysis is less modular than our first. Although it does not need to examine clients of a method, it relies on a closed-world assumption relative to overriding: the analysis is valid only if the library designers do not expect a method to be overridden with more specific uses of its arguments.) These methods are a superset of the over-specified methods found in the first analysis.

We ran our inference again on the code bases with modified method arguments. The highlighted rows of Figure 10 show the results of our second analysis. The percentage of variant versus invariant classes changes rather dramatically once method signatures are properly specified. Overall, 46% of classes are variant in at least one type parameter. This translates to 41% of variant type parameters. The percentage of unnecessary wildcards and over-specified methods (in the original code) increased to 52% and 41%, respectively. Note that since the over-specified method column is not all over-specified methods, its values may stay the same across two versions (as in JScience and GNU Trove).

Bivariance and Backward Compatibility. As discussed in Section 3, if the definition-site variance of a generic only has a recursive constraint, that is, it depends only on its own variance, then the most liberal form of variance we could assign is bivariance.

It is questionable why anyone would choose to parameterize a class or interface with a type parameter at all, if the parameter is not used in types of references read from or written to (other than in a constructor call). However, in our analysis, we find that 3% of classes and interfaces are indeed defined in this manner (5% in our second analysis).

We inspected these generics to find whether there is an obvious reason for this curious practice. We found that the invariant type parameters appear to be used for enforcing a documentation discipline, and that the designers of these generics are in fact relying on Java’s default invariance to enforce this discipline. For instance, in the reference implementation for JSR 275 (Measures and Units) [11], included with the JScience library [10], a group of 11 classes and interfaces are collectively bivariant in a type parameter, `Q extends Quantity`. In the definition of `Unit<Q extends Quantity>`, for example, the type parameter `Q` appears nowhere other than as the type argument to `Unit<Q>`. Closer inspection of the code shows that `Quantity` is extended by 43 different subinterfaces, such as `Acceleration`, `Mass`, `Torque`, `Volume`, etc. It appears that the authors of the library are actually relying on the invariant subtyping of Java generic classes, to ensure that `Unit<Acceleration>` is never used as `Unit<Mass>`, etc.! This kind of type parameters are sometimes referred to as “shadow type parameters.”

Our system by default infers the most general variance possible, which, in this case, is bivariance. However, to make our type system a smooth, backward compatible extension of Java without extra syntax, we offer a compiler option that designates whether to infer bivariance or *invariance*, if a generic has only recursive variances. The impact of this change is only in the group of generics that recursively depend on each other’s variance and nothing else.

Recall that our type system is designed to allow as general use of generics as possible, thus some previously non-valid programs will compile (but never cause a dynamic type error). The above is the only instance we found of a programming pattern that relies on the artificially restricted type system, and it can easily be treated heuristically.

6 Related Work

Definition-site variance was first investigated in the late 80’s [1, 5, 9], when parametric types were incorporated into object-oriented languages. It has experienced a resurgence in recent years, as newer languages such as Scala [20] and C# [15] chose it as means to support variant subtyping. Perhaps surprisingly, with such a long history, it has only recently been formalized and proven sound [13]. Previous work focused on the safety characteristics of definition-site variance, not on how such variance can be inferred (other than for basic, non-recursive types). There has never been a study of how to determine definition-site variance in

the face of recursive type definitions—which are ubiquitous in object-oriented programs. This paper is the first, to our knowledge, to classify the four types of recursive variant constraints that can occur, and how they can be resolved.

Use-site variance was introduced by Igarashi and Viroli [16], in response to the rigidity in class definitions imposed by definition-site variance. The elegance and flexibility of the approach evoked a great deal of enthusiasm, and was quickly introduced into Java [23]. The same flexibility also proved challenging to both researchers and practitioners. The soundness of wildcards in Java has only recently been proven [6], and the implementation of wildcards has been mired in issues [8, 22]. We do not address these issues directly. Instead, our work minimizes the necessity of wildcards in general, by inferring definition-site variance. Furthermore, no one has conducted a study on exactly how much of the flexibility we really need in practice. We provide the first study that shows that roughly half of the generics are defined with single variance, without even being designed for it. Programmers should be allowed to take advantage of this knowledge.

Kiezun et al. [18] offer an approach to inferring variance annotations. That work, however, is a heuristic-based approach, relying on *whole program analysis*. There is no separation between the definition of a generic and its uses. Thus, the results of the approach are entirely dependent on the code available for analysis, and the heuristics chosen for that code. Given the problem that the authors were solving—automatically inferring type parameters for unparameterized classes and interfaces—this is a perfectly valid approach, and it produced impressive results. However, our work presents a modular, deterministic, and principled approach to combining definition-site variance inference with use-site variance.

7 Conclusions

We show that the need for the flexibility of use-site variance is somewhat overrated, and the rigidity of definition-site variance is unwarranted. We combine the benefits of both and mitigate the burdens by *modularly inferring* the definition-site variance, and allowing use-site variance annotations where necessary. Thus, our approach resolves questions that are central in the design of any object-oriented language. Our work is the first to fully study the complexities involved in recursive class definitions with respect to variance inference, and show how they can be resolved. We show that combining definition-site and use-site variance allows us to infer more general types than either approach alone. Our Java implementation is completely backward compatible, support for separate compilation is easy with bytecode annotations, and no change to the JVM is necessary.

References

1. P. America and F. van der Linden. A parallel object-oriented language with inheritance and subtyping. In *OOPSLA/ECOOP '90: Proc. of the European Conf. on object-oriented programming on Object-oriented programming systems, languages, and applications*, 1990.

2. Apache Software Foundation. *Apache Commons-Collections Library*. <http://commons.apache.org/collections/>. Accessed Dec. '08.
3. J. Bloch. *The Closure Controversy*, http://www.javapolis.com/confluence/download/attachments/32918/C_13_08_02.pdf. Accessed Dec. 2008.
4. K. Boumilion and J. Levy. *Google Collections Library*, <http://code.google.com/p/google-collections/>. Accessed Dec. '08.
5. G. Bracha and D. Griswold. Strongtalk: typechecking smalltalk in a production environment. In *OOPSLA '93: Proc. of the Conf. on Object-oriented programming systems, languages, and applications*, 1993.
6. N. Cameron, S. Drossopoulou, and E. Ernst. A model for java with wildcards. In *ECOOP '08: Proc. of the 22nd European Conf. on Object-Oriented Programming*, 2008.
7. R. Cartwright and J. Guy L. Steele. Compatible genericity with run-time types for the java programming language. In *OOPSLA '98: Proc. of the 13th ACM SIGPLAN Conf. on Object-oriented programming, systems, languages, and applications*, 1998.
8. W.-N. Chin, F. Craciun, S.-C. Khoo, and C. Popeea. A flow-based approach for variant parametric types. In *OOPSLA '06: Proc. of the 21st Conf. on Object-oriented programming languages, systems, and applications*, 2006.
9. W. Cook. A proposal for making eiffel type-safe. In *ECOOP '89: Proc. of the 3rd European Conf. on Object-Oriented Programming*, 1989.
10. J.-M. Dautelle et al. *JScience Website*, <http://jscience.org/>. Accessed Dec. '08.
11. J.-M. Dautelle and W. Keil. *JSR-275: Measures and Units*, <http://www.jcp.org/en/jsr/detail?id=275>. Accessed Dec. '08.
12. M. Day, R. Gruber, B. Liskov, and A. C. Myers. Subtypes vs. where clauses: constraining parametric polymorphism. In *OOPSLA '95: Proc. of the tenth annual Conf. on Object-oriented programming systems, languages, and applications*, 1995.
13. B. Emir, A. Kennedy, C. Russo, and D. Yu. Variance and generalized constraints for C# generics. In *ECOOP '06: Proc. of the European Conf. on Object-Oriented Programming*, 2006.
14. E. Friedman and R. Eden. *GNU Trove High-performance Collections Library for Java*. <http://trove4j.sourceforge.net/>. Accessed Dec. '08.
15. A. Hejlsberg, S. Wiltamuth, and P. Golde. *C# Language Specification*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, 2003.
16. A. Igarashi and M. Viroli. On variance-based subtyping for parametric types. In *ECOOP '02: 16th European Conf. on Object-Oriented Programming*, 2002.
17. A. Igarashi and M. Viroli. Variant parametric types: A flexible subtyping scheme for generics. *ACM Trans. Program. Lang. Syst.*, 28(5):795–847, 2006.
18. A. Kiezun, M. Ernst, F. Tip, and R. Fuhrer. Refactoring for parameterizing java classes. In *ICSE '07: Proc. of the 29th Intl. Conf. on Software Engineering*, 2007.
19. B. Meyer. *Eiffel: The Language*. Prentice Hall, 1991.
20. M. Odersky. *The Scala Language Specification*, <http://scala.epfl.ch/docu/files/ScalaReference.pdf>. 2008. Accessed Dec. 2008.
21. A. Salcianu. *Java Program Analysis Utilities Library*. <http://jpaul.sourceforge.net/>. Accessed Dec. '08.
22. D. Smith and R. Cartwright. Java type inference is broken: can we fix it? In *OOPSLA '08: Proc. of the 23rd ACM SIGPLAN Conf. on Object oriented programming systems languages and applications*, 2008.
23. M. Torgersen, C. P. Hansen, E. Ernst, P. von der Ahe, G. Bracha, and N. Gafter. Adding wildcards to the java programming language. In *SAC '04: Proc. of the 2004 Symposium on Applied Computing*, 2004.